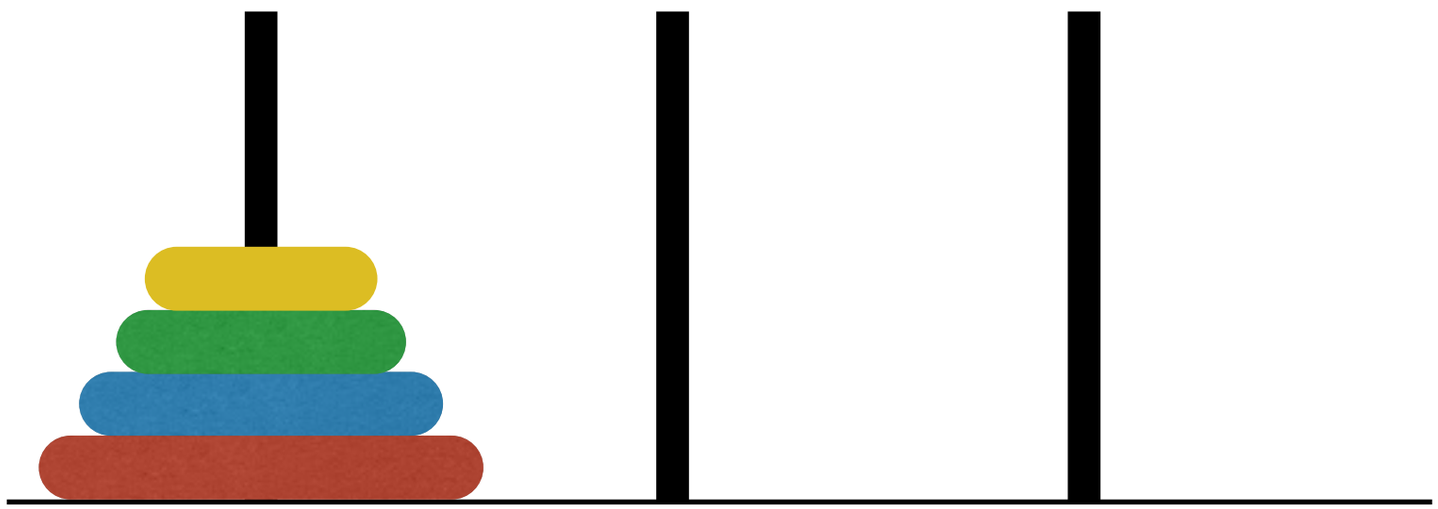
Data Structures in Java

Lecture 4: More Recursion.
Abstract Data Types. Array Lists.

9/19/2016

Daniel Bauer

The Towers of Hanoi

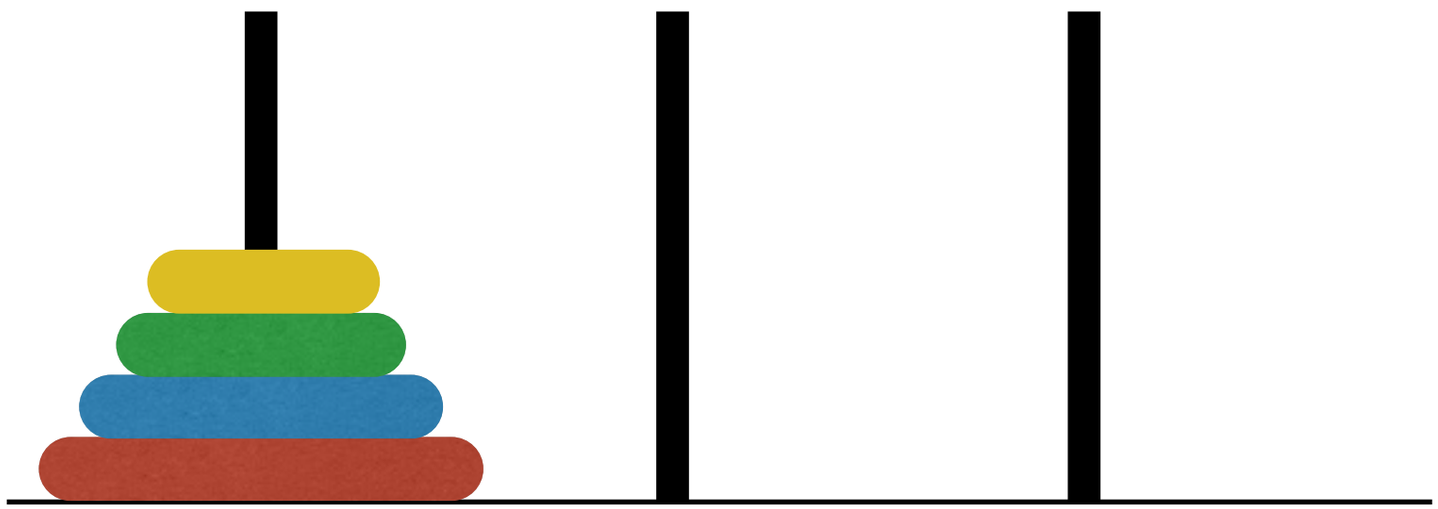


A B C

Goal: Move all disks to the right peg

Moves: Take any disk on top of a stack and move it to the top of another stack. No disk may be placed on a smaller disk.

The Towers of Hanoi

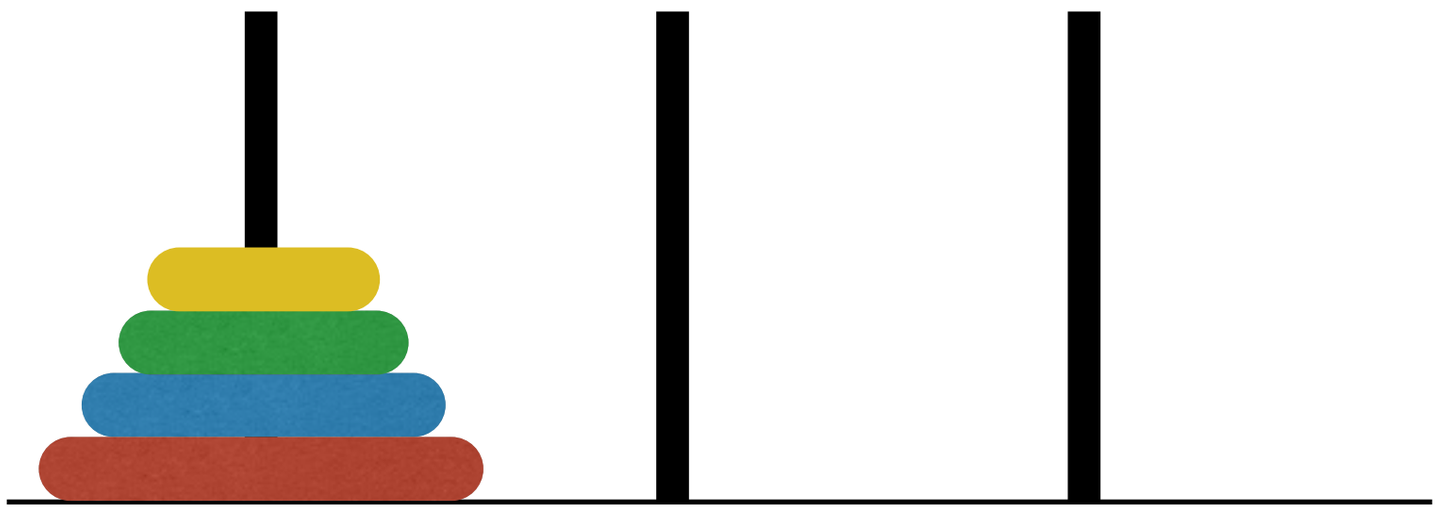


A B C

Insight: To move 4 disks from A to C

1. move top three disks from A to B
2. move fourth disk to C
3. move top three disks from B to C

The Towers of Hanoi

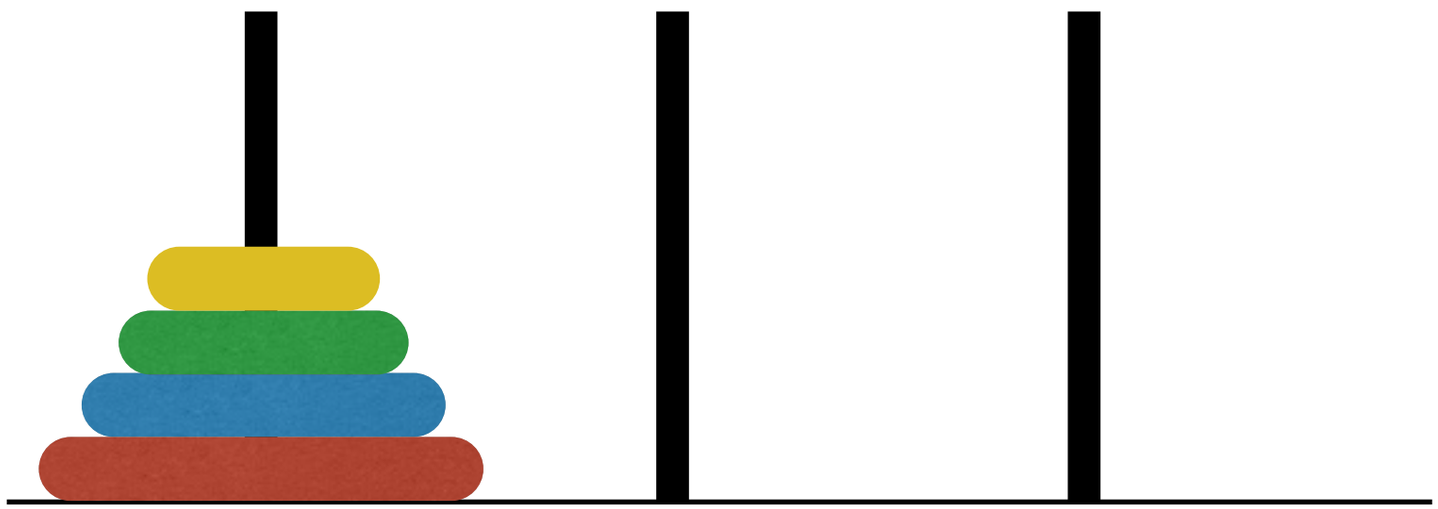


A B C

Insight: To move 3 disks from A to B

1. move top two disks from A to C
2. move third disk to B
3. move top two disks from C to B

The Towers of Hanoi



A B C

Insight: To move 2 disks from A to C

1. move top one disks from A to B
2. move third disk to C
3. move top one disks from B to C

The Towers of Hanoi

Algorithm (sketch)

To move *n* disks from A to C
1. move top *n-1* disks from A to B
2. move *n*-th to C
3. move top *n-1* disks from B to C

A = source peg
C = target peg
B = “help” peg (to temporarily store disks)

Peg labels change in each recursive call.

The Towers of Hanoi

To move *n* disks from A to C

1. move top *n-1* disks from A to B
2. move *n*-th to C
3. move top *n-1* disks from B to C

$$T(N) = 2 \cdot T(N-1) + 1$$

$$T(1) = 1$$

Need to solve this recurrence relation!

Analyzing the Towers of Hanoi Recurrence

$$T(N) = 2 \cdot T(N-1) + 1$$

$$= 2 \cdot (2 \cdot T(N-2) + 1) + 1 = 2^2 \cdot T(N-2) + 2 + 1$$

$$= 2^2 \cdot (2 \cdot (N-3) + 1) + 2 + 1$$

$$= 2^3 \cdot T(N-3) + 2^2 + 2 + 1 = 2^3 \cdot T(N-1) + 2^2 + 2^1 + 2^0$$

$$= 2^{N-1} T(1) \cdot 2^{N-2} + 2^{N-3} + \cdots + 2^0$$

$$= \sum_{j=0}^{N-1} 2^j = 2^N - 1$$

base case: $T(1) = 1$

geometric series

The End of The World

Legend says that, at the beginning of time, priests were given a puzzle with 64 golden disks. Once they finish moving all the disks according to the rules, the wold is said to end.

If the priests move the disks at a rate of 1 disk/second, how long will it take to solve the puzzle?

$$T(N) = 2^N - 1 = \Theta(2^N)$$

$$\begin{aligned} 2^{64} - 1 &= 18,446,744,073,709,551,615 && \text{seconds} \\ &= 307,445,734,561,825,860 && \text{minutes} \\ &= 213,503,982,334,601 && \text{days} \\ &= 584,942,417,355 && \text{years} \end{aligned}$$

Overview

- 1. Recursion
- **2. Abstract Data Types**
- 3. Lists, ArrayList

Data Types

- Basic data types: booleans, bytes, integers, floats, characters…

- Simple abstractions: arrays, `String`

- More complex data types (**this class**): Lists, Stacks, Trees, Sets, Graphs

Abstract Data Types

- An Abstract Data Type (ADT) is an collection of data together with a set of operations.

- ADT specification *does not mention how* operations are implemented.

- Example:

  - `Set` ADT might provide “add”, “remove”, “contains”, “union”, and “intersection” operations.

ADTs vs. Data Structures

- A data *type* is a well-defined collection of data with a well-defined set of operations on it.

- A *data structure* is an actual implementation of a particular abstract data type.

ADTs in Java

- ADTs can be specified as interfaces. Interfaces define behavior, but say nothing about implementation or performance issues

- ADTs can be implemented as classes. Careful class design hides implementation details from users, enabling “plug and play”.

    - Encourage re-usability of components!

ADTs in Java

- Example: Java `Strings`
  - We can call methods such as `length()` and `concat(String str)`.
  - We don't have to know how Strings are stored in memory or how methods are implemented.
  - How many bytes does it take to represent a character?

Some ADTs we will study

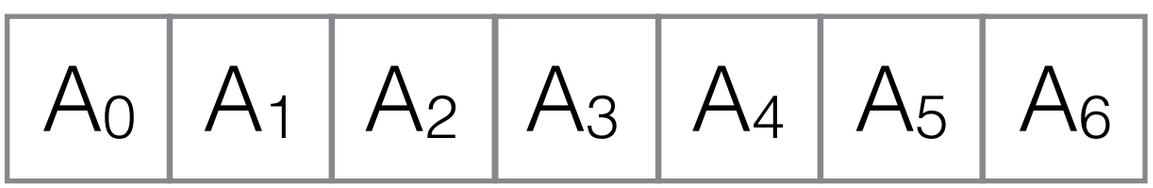
- Lists
- Stacks
- Queues
- Priority Queues (Heaps)
- Search Trees
- Graphs

Overview

- 1. Recursion
- 2. Abstract Data Types
- **3. Lists, ArrayList**

The List ADT

- A list *L* is a sequence of *N* objects A_0, A_1, A_2, ..., A_{N-1}
- *N* is the length/size of the list. List with length N=0 is called the *empty list.*
- A_i *follows/succeeds* A_{i-1} for i > 0.
- A_i *precedes* A_{i+1} for i < N.



Typical List Operations

- `void printList()`
- `void makeEmpty()`
- `int size()`
- `Object findKth(k) / get(k)`

- `boolean insert(x, k), append(x)`
- `boolean remove(k)`
- `int find(x) / indexOf(x)`
- `Object next()`
- `Object previous()`
- `void removeCurrent()`

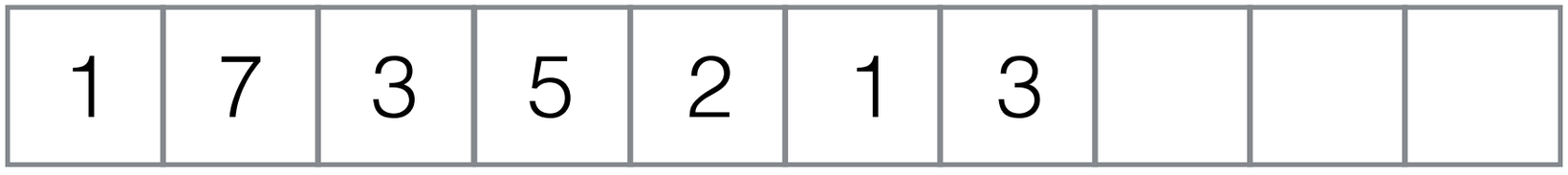
Array Lists

- Just a thin layer wrapping an array.

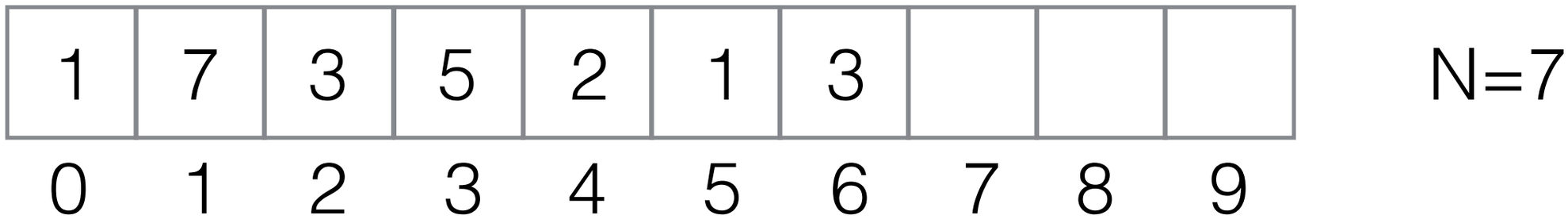
```
public class ArrayList {

    public static final int DEFAULT_CAPACITY = 10;
    private int theSize;
    private Integer[] theItems;

}
```

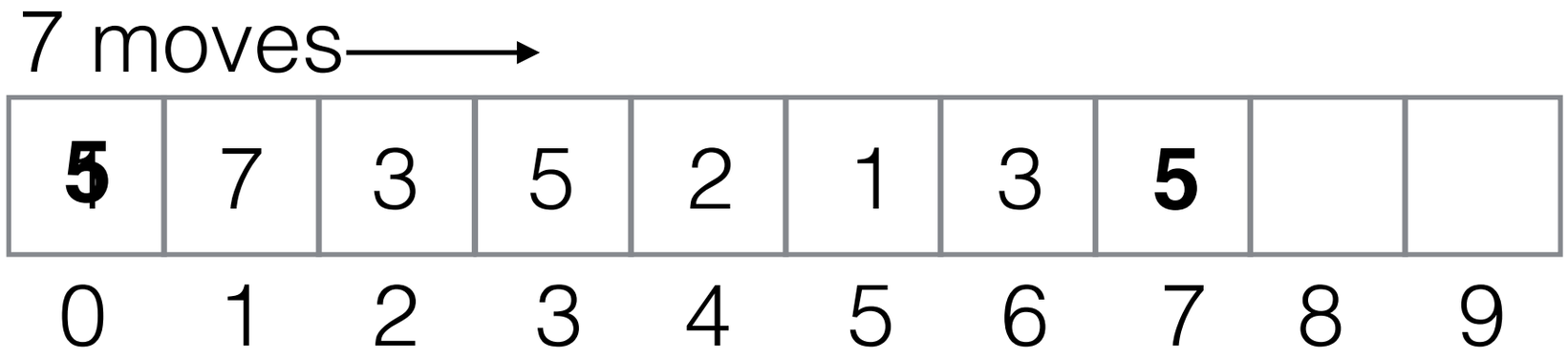


Running Time for Array List Operations



Operation	Number of Steps
printList	N
find(x)	N
findKth(k)	1
insert(x,k)	?
remove(x)	?

Array List: Insert/Remove



insert(5,7): 1 step

remove(7): 1 step

best case

insert(5,0): 7 steps

remove(0): O(N)

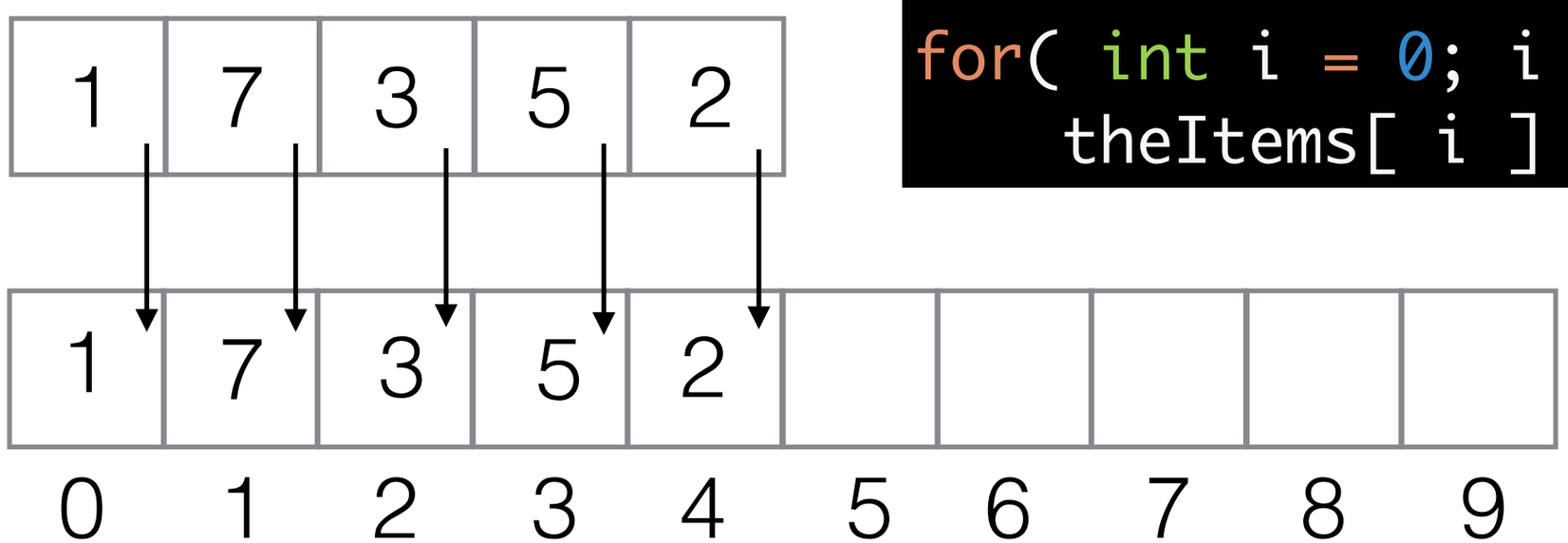
worst case

insert(x,k)	N
remove(x)	N

Expanding Array Lists

- What if we are running out of space during `append/insert`
- first copy all elements into a new array of sufficient size

```
newCapacity = arr.length * 2;
Integer[ ] old = theItems;
theItems = new Integer[newCapacity];
for( int i = 0; i < size( ); i++ )
    theItems[ i ] = old[ i ];
```



(See Java Code)